

기계화 명세 기반 이더리움 합의 클라이언트 비교 테스트

**SpecTrum: Specification-Guided Differential Fuzzing
for Ethereum Consensus Clients**

정석훈, 단경민, 류석영, 황성재 (ASE'26)

이더리움 ∈ 블록체인 ∈ 분산시스템

- **분산시스템:** 여러 개체가 마치 하나의 개체처럼 움직이게 하는 것 (동기화, 업무분배)
- **블록체인:** 분산시스템 + 영속성 + 열린 네트워크 (악성 개체)

이더리움의 두 가지 방어기제

1. 합의 알고리즘: 악성 개체로부터 방어

- 비트코인(PoW)은 채굴로 지분(voting power)을 획득
- 이더리움(PoS)은 입장료로 동등한 지분 획득, 위반사항에 따라 삭감

2. 클라이언트 다양성: 구현체의 결함으로부터 방어

- 하나의 명세를 서로 다른 여러 구현체(클라이언트)가 구현
 - 명세는 Python / 클라이언트는 Rust, Go, Java, TypeScript, Nim
- 유저는 클라이언트를 자유롭게 선택해 **validator**로 합의 네트워크에 참여

합의 알고리즘 + 클라이언트 다양성

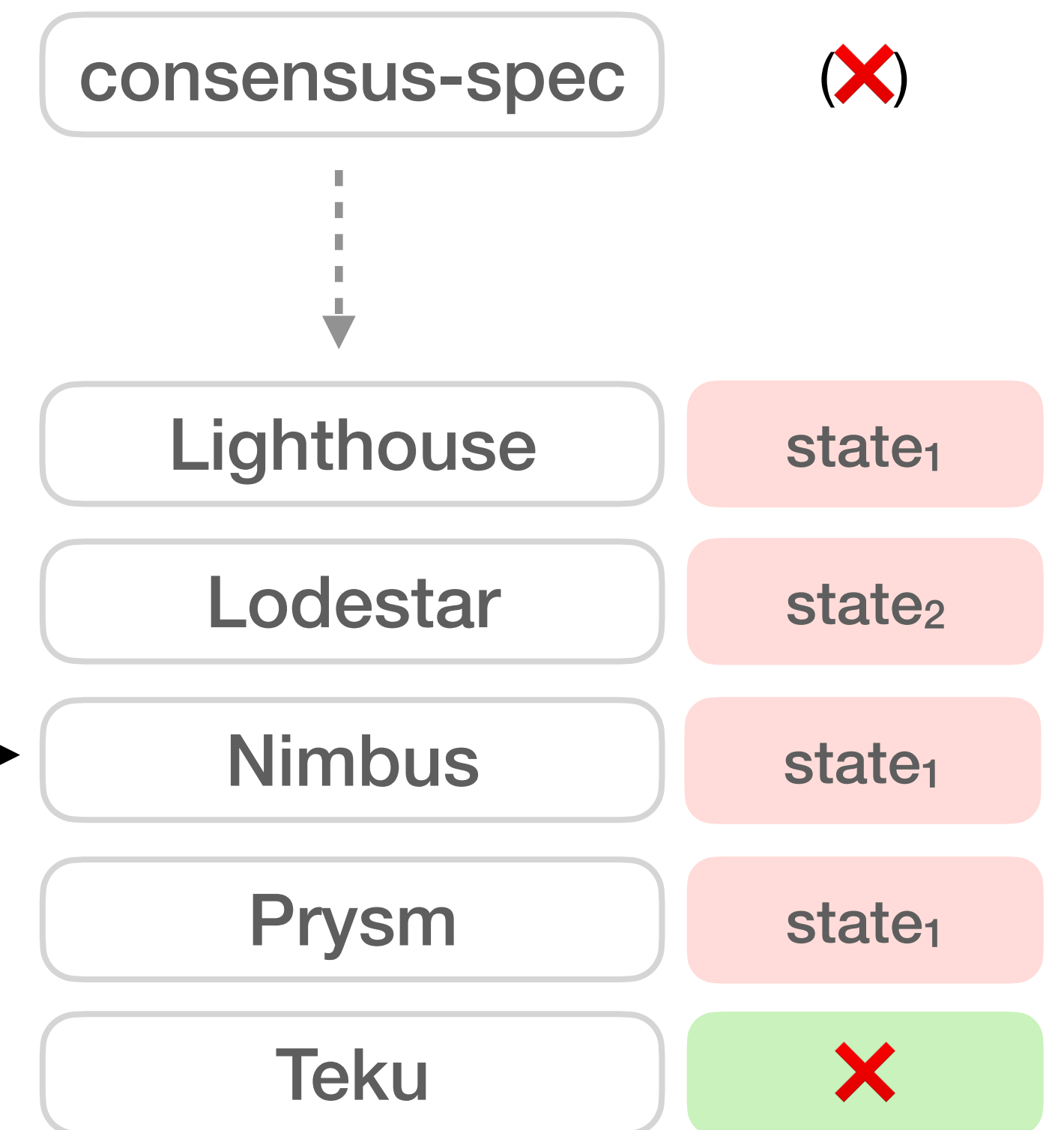
합의 알고리즘: $f(\text{state}_{\text{pre}}, \text{block}) = \text{state}_{\text{post}}$ | ❌

실행가능한 파이썬 명세 (consensus-spec) + 5개의 구현체

오류를 일으키는 validator 개수에 따라 네트워크 안정성 위협

(s , b) →

실행가능한 명세로 의도된 결과를 확인할 수 있고,
제공된 테스트(spectests)에서 명세와 동일한 결과를 내야 함
그러나, 개발자는 명세 코드를 보고 잘 이해해서 번역해야 함



예시

consensus-spec (Python): weigh_and_justify_finalization (simplified)

```
1 if prev_bal * 3 >= total_bal * 2:
2     state.curr_jcp = Checkpoint(prev_ep, ...)
3 if curr_bal * 3 >= total_bal * 2:
4     state.curr_jcp = Checkpoint(curr_ep, ...)
```

예시

consensus-spec (Python): weigh_and_justify_finalization (simplified)

```
1 if prev_bal * 3 >= total_bal * 2:  
2     state.curr_jcp = Checkpoint(prev_ep, ...)  
3 if curr_bal * 3 >= total_bal * 2:  
4     state.curr_jcp = Checkpoint(curr_ep, ...)
```

◀ 분기 조건이 중첩 가능

예시

consensus-spec (Python): weigh_and_justify_finalization (simplified)

```
1 if prev_bal * 3 >= total_bal * 2:  
2     state.curr_jcp = Checkpoint(prev_ep, ...)  
3 if curr_bal * 3 >= total_bal * 2:  
4     state.curr_jcp = Checkpoint(curr_ep, ...)
```

◀ 분기 조건이 중첩 가능

◀ mutation으로 필드 덮어쓰기

예시

consensus-spec (Python): weigh_and_justify_finalization (simplified)

```
1 if prev_bal * 3 >= total_bal * 2:  
2     state.curr_jcp = Checkpoint(prev_ep, ...)  
3 if curr_bal * 3 >= total_bal * 2:  
4     state.curr_jcp = Checkpoint(curr_ep, ...)
```

◀ overflow 발생 가능

◀ 분기 조건이 중첩 가능

◀ mutation으로 필드 덮어쓰기

예시

consensus-spec (Python): weigh_and_justify_finalization (simplified)

```
1 if prev_bal * 3 >= total_bal * 2:
2     state.curr_jcp = Checkpoint(prev_ep, ...)
3 if curr_bal * 3 >= total_bal * 2:
4     state.curr_jcp = Checkpoint(curr_ep, ...)
```

◀ overflow 발생 가능

◀ 분기 조건이 중첩 가능

◀ mutation으로 필드 덮어쓰기

*"State transitions that trigger an **unhandled exception** (e.g. a failed assert or an out-of-range list access) or an uint64 **overflow or underflow** are considered invalid."*

방법: ① 명세 풀어써주기

consensus-spec (Python): weigh_and_justify_finalization (simplified)

```
1 if prev_bal * 3 >= total_bal * 2:
2     state.curr_jcp = Checkpoint(prev_ep, ...)
3 if curr_bal * 3 >= total_bal * 2:
4     state.curr_jcp = Checkpoint(curr_ep, ...)
```

◀ overflow 발생 가능

◀ 분기 조건이 중첩 가능

◀ mutation으로 필드 덮어쓰기

Consensus-SpecTec: WeighJustificationAndFinalization (simplified)

```
1 rule WeighJustAndFin/justify_prev:
2     state, total_bal, prev_bal, curr_bal
3     ~> state[.CURR_JCP = { prev_ep, ... }]
4 -- if total_bal <= UINT64_MAX/2
5 -- if prev_bal * 3 >= total_bal * 2
6 -- if ~(curr_bal * 3 >= total_bal * 2)
```

방법: ① 명세 풀어써주기

consensus-spec (Python): weigh_and_justify_finalization (simplified)

```
1 if prev_bal * 3 >= total_bal * 2:
2     state.curr_jcp = Checkpoint(prev_ep, ...)
3 if curr_bal * 3 >= total_bal * 2:
4     state.curr_jcp = Checkpoint(curr_ep, ...)
```

◀ overflow 발생 가능

◀ 분기 조건이 중첩 가능

◀ mutation으로 필드 덮어쓰기

Consensus-SpecTec: WeighJustificationAndFinalization (simplified)

```
1 rule WeighJustAndFin/justify_prev:
2     state, total_bal, prev_bal, curr_bal
3     ~> state[.CURR_JCP = { prev_ep, ... }]
4 -- if total_bal <= UINT64_MAX/2
5 -- if prev_bal * 3 >= total_bal * 2
6 -- if ~(curr_bal * 3 >= total_bal * 2)
```

◀ 분기 조건에 따른 실행경로 분리

방법: ① 명세 풀어써주기

consensus-spec (Python): weigh_and_justify_finalization (simplified)

```
1 if prev_bal * 3 >= total_bal * 2:
2     state.curr_jcp = Checkpoint(prev_ep, ...)
3 if curr_bal * 3 >= total_bal * 2:
4     state.curr_jcp = Checkpoint(curr_ep, ...)
```

◀ overflow 발생 가능

◀ 분기 조건이 중첩 가능

◀ mutation으로 필드 덮어쓰기

Consensus-SpecTec: WeighJustificationAndFinalization (simplified)

```
1 rule WeighJustAndFin/justify_prev:
2     state, total_bal, prev_bal, curr_bal
3     ~> state[.CURR_JCP = { prev_ep, ... }]
4 -- if total_bal <= UINT64_MAX/2
5 -- if prev_bal * 3 >= total_bal * 2
6 -- if ~(curr_bal * 3 >= total_bal * 2)
```

◀ mutation대신 새 state값 반환

◀ 분기 조건에 따른 실행경로 분리

방법: ① 명세 풀어써주기

consensus-spec (Python): weigh_and_justify_finalization (simplified)

```
1 if prev_bal * 3 >= total_bal * 2:
2     state.curr_jcp = Checkpoint(prev_ep, ...)
3 if curr_bal * 3 >= total_bal * 2:
4     state.curr_jcp = Checkpoint(curr_ep, ...)
```

◀ overflow 발생 가능

◀ 분기 조건이 중첩 가능

◀ mutation으로 필드 덮어쓰기

Consensus-SpecTec: WeighJustificationAndFinalization (simplified)

```
1 rule WeighJustAndFin/justify_prev:
2     state, total_bal, prev_bal, curr_bal
3     ~> state[.CURR_JCP = { prev_ep, ... }]
4 -- if total_bal <= UINT64_MAX/2
5 -- if prev_bal * 3 >= total_bal * 2
6 -- if ~(curr_bal * 3 >= total_bal * 2)
```

◀ mutation대신 새 state값 반환

◀ overflow 조건 삽입

◀ 분기 조건에 따른 실행경로 분리

방법: ② 조건 커버리지 측정

Consensus-SpecTec: WeighJustificationAndFinalization (simplified)

```
1 rule WeighJustAndFin/justify_prev:
2   state, total_bal, prev_bal, curr_bal
3   ~> state[.CURR_JCP = { prev_ep, ... }]
4 -- if total_bal <= UINT64_MAX/2
5 -- if prev_bal * 3 >= total_bal * 2
6 -- if ~(curr_bal * 3 >= total_bal * 2)
```

공식 end-to-end test 581개의

각 -- if 조건 성공/실패여부 추적

	True / False
◀	21 / 0
◀	16 / 5
◀	9 / 7

true값을 일으키는 seed를 변이해

false값을 생성

방법: ③ 실패 누락 조건을 겨냥해서 테스트 생성

Consensus-SpecTec: WeighJustificationAndFinalization (simplified)

```
1 rule WeighJustAndFin/justify_prev:  
2   state, total_bal, prev_bal, curr_bal  
3   ~> state[.CURR_JCP = { prev_ep, ... }]  
4 -- if total_bal <= UINT64_MAX/2  
5 -- if prev_bal * 3 >= total_bal * 2  
6 -- if ~(curr_bal * 3 >= total_bal * 2)
```

입력값 (**state**, **block**)의 필드 경로에 대한
symbolic execution

◀ `state.VAL[1].BAL + state.VAL[3].BAL`
 `<= UINT64_MAX/2`



From: `state.VAL[1].BAL`,

To:

> (`UINT64_MAX/2 - state.VAL[3].BAL`)

결과: ① 불일치 발견

Category		Implicit
Precondition Violation	●●	(1 / 2)
Lossy Decoding	■ ■ ■ ■ ■ ● ■ ■ ■ ■	(8 / 9)
Configuration Inconsistency	●	(0 / 1)
Numeric Overflow	■ ● ● ● ●	(1 / 5)
State-update Inconsistency	●	(0 / 1)
Invariant Enforcement	● ● ● ● ● ●	(0 / 6)

■ : 유형 A (10 / 24)
● : 유형 B (14 / 24)

5개 구현체 × 13,799개의 테스트
→ **24개**의 불일치, 전부 보고 완료

- 유형 A (10): 역할 구분없이 항상 발생
- 유형 B (14): block을 제안할 때 발생

결과: ② 커버리지 비교

Implementation	Branch Coverage
consensus-spec	152 → 154 (+2) / 358
Lighthouse	177 → 187 (+10) / 486
Lodestar	275 → 307 (+32) / 422
Nimbus	804 → 879 (+75) / 19270
Prysm	188 → 260 (+72) / 1226
Teku	447 → 478 (+31) / 1258

조건 커버리지: +60
126/204 (61.8%) → 186/204 (**91.2%**)

각 구현체의 커버리지 변화는 미미

54개/60개 조건이 새로 삽입한 조건

13개/24개 불일치를 삽입한 조건으로 발견

요약

명세의 조건이 함축적이면

- 구현체도 조건을 빠뜨리기 쉽고
- 조건이 없기에 커버리지로 드러나지 않으며
- 해당 조건을 검사하는 테스트가 없을 확률이 높음

본 연구에서는

- 명세의 함축적인 조건을 **풀어써줌**으로써
- 빠진 조건에 해당하는 **테스트를 생성**하고
- 명세 및 구현체끼리 서로 다르게 구현한 **불일치를 발견**